CS5013 Mid-Evaluation Deliverable: Principles & Practices

Unified Lab Project & Idea Tracking Platform

Team 47: Priyal Patel (CS24B047) & Sujal Jayant Kumar (DA24B026)

October 10, 2026

We are building a web app for Raman Lab (DSAI, IIT Madras) that tracks projects and ideas, saves the paper links people share in Slack, and lets you search across all of it. It is a Spring Boot (Java 21) app on PostgreSQL. Sujal owns sign-in, the data layer, projects and ideas; Priyal owns Slack ingestion, the paper archive and search.

1 Prompt Engineering

We tried to incorporate these practices learnt from the course to make our prompting for AI assistance better. Real examples are in Appendix A.

1. **Goal, limits, and how we will check.** Every prompt says what we want, what is out of scope ("do not start the worker or the UI yet"), and how the result is verified ("run `mvn verify` and report the real counts; do not say tests passed unless you ran them").
2. **Plan, then scaffold, then fill in.** We ask the AI for a plan first (files to touch, dependencies, test strategy) and read it before any code is written. Sujal wrote a scaffolding guide, and the AI was prompted to fill in each feature sequentially on its own branch.
3. **Examples and edge cases inside the prompt.** Instead of "add tests", we list specific cases for the AI to handle: a known DOI, an unknown one, a year-only date. We feed real responses from the Crossref and arXiv APIs into the prompt so the assistant works from real data.
4. **Say who it is for, and in what style.** If the output is for another person we name them: pull request descriptions are written for the teammate who will review them. When we want a review rather than code, we set a role ("act as a strict reviewer of this diff").
5. **Check, do not trust.** The assistant can sound sure and be wrong. When an AI tool explained a failed CI run, it suggested `continue-on-error` (which hides failures). We rejected this hallucinated shortcut and forced it to write a proper retry loop.

2 AI-Assisted Debugging

We use the AI as a diagnostic partner through a strict routine: **(1)** isolate the minimum failing test case; **(2)** paste the full stack trace and the exact file context into the prompt; **(3)** ask the AI to suggest potential hypotheses and fixes rather than writing a blind patch; **(4)** we evaluate the options, pick the root cause, and prove it; **(5)** ask the AI to scan for the same vulnerability elsewhere; **(6)** leave a test or rule behind. Examples:

- **A test that failed only sometimes.** `SlackTablesTest` intermittently failed with a unique constraint violation. Sujal's review on PR #10 pointed to the cause: `IngestionReplayTest` was missing `@Transactional`. We gave the AI the trace and both tests to confirm the collision, fix it, and then prompted it to check every other database test for the same leak.
- **A CI run that failed for the wrong reason.** The log ended in an HTTP 502 from Maven Central. We pasted the error to the AI to understand the root cause. When it suggested a bad workaround, we corrected it and prompted it to generate a minimal YAML retry loop for the dependency download.
- **A bug caught by reading real output.** Reviewing the link extractor, Springer links ending in `/fulltext.html` were stored incorrectly. We described the observed vs. expected behavior to the AI and asked it to refactor the regex to normalize all forms into one DOI link, alongside generating a new test case.

3 Version Control (Git)

- **AI-suggested branch names.** We prompt the AI with our issue descriptions to generate concise, standardized branch names (`schema`, `paper-resolvers`). When features depend on another, we *stack* the pull requests.
- **Peer reviews & CI.** Every PR is reviewed by the other person before it merges. GitHub Actions runs `mvn verify` on every PR so a red build is visible before anyone merges.
- **AI-drafted commits and PRs.** We pipe our `git diff` into the assistant and prompt it to draft structured PR descriptions and commit messages. We then manually append the contextual "why" ourselves to ensure institutional knowledge is preserved.
- **The assistant never commits.** It leaves changes in the working tree and drafts the exact git commands for managing our stacked branches; we read the diff, stage, and commit ourselves.
- **Prompts and work are versioned with the code.** `Prompt_logs/` holds our cleaned-up prompts and `Work_logs/` holds our diaries, ensuring our AI conversations are tracked in the same history as the code they produced.

4 Testing, Refactoring & Documentation

Testing. We maintain an actively growing suite of automated JUnit 5 tests. We heavily rely on the AI to quickly generate tedious edge-case permutations (like the 17 cases for the link extractor) and WireMock stubs based on our recorded API responses, while we manually verify the assertions.

What	How we test it
Database rules	Real PostgreSQL (Zonky embedded). The AI drafts the `@Transactional` rollbacks.
Outside services	WireMock serves *real* responses. AI helps scaffold the error state tests (503s, malformed IDs).
Web layer	MockMvc with AI-generated simulated Google sign-in filters and redirects.

Refactoring. Refactoring happens in small steps with AI assistance keeping the tests green. When we needed resolvers to share one interface and an `@Order`, we asked the AI to draft

the boilerplate extraction so the worker remained untouched. We act as the final reviewers, prioritizing readable code over clever logic, frequently rejecting over-engineered AI suggestions.

Documentation. We pass our finalized code to the assistant to write the first draft of our `README` updates and Javadoc. We then manually review and correct each method's documentation so it accurately reflects the business logic. `CLAUDE.md` explicitly dictates these AI rules to the model on every new session.

A Sample Prompts

Cleaned-up versions of prompts used during development, stored in `Prompt_logs/`.

A1. Plan first, no code yet

(context, scope, ask only what you cannot infer)

> Based on the repository and handoff documents, propose a small implementation plan for the Slack-to-database ingestion flow and the read-only papers API. List the files to add or change, how the Slack client will be used, how duplicate events will be handled, and how each part will be tested. Keep the work within Phase 1 and avoid starting the resolver ladder or search features. Ask only about decisions that cannot be inferred from the repo.

A2. Debugging with evidence

(exact error, suspected cause, a way to confirm, a fence)

> PR #10 has a review comment: `SlackTablesTest.duplicatePaperUrlIsRejected` sometimes fails with *duplicate key value violates unique constraint "paper_normalized_url_key"* for `https://arxiv.org/abs/1706.03762`. The review says `IngestionReplayTest` is not `@Transactional`, so its papers stay in the shared database. Confirm that by reading both tests, `PostgresTestBase` and `IngestionService`. Then fix it the way the other database tests do and remove any manual clean-up the annotation makes redundant. Keep the change inside the test files. Afterwards, check every other database test for the same problem and run the suite in reverse order.